

Data Representation

DSC 204A: Scalable Data Systems

Haojian Jin

Where we are in the class?

Foundations of Data Systems

- **Data representation & encoding**
- Basics of Computer Organization
- Operating Systems
- Storage & Memory Hierarchy
- Cache

Scaling & Distributed Systems

- Encoding & evolution
- Distributed Processing
- Programming model

Machine Learning Systems

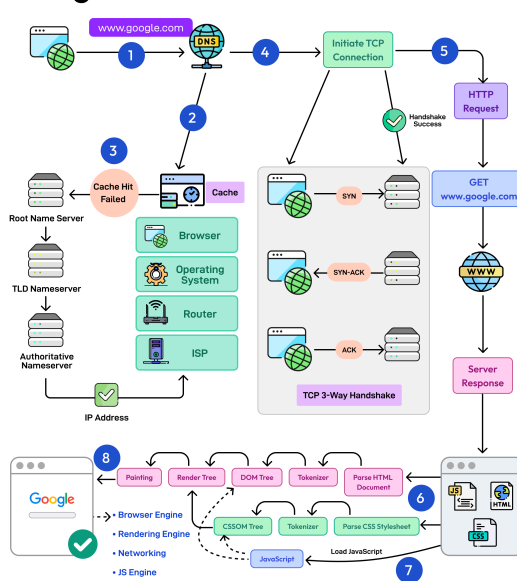
- ML Systems
- Batch & Stream Processing

Understand CS/DS/AI from the Users' Perspective

- What will happen when you type google.com in your browser?
- What LLM models can you run on your local computer?
- What happens when you ask Claude/Codex to write a program for you?
- What happens when Claude Code says "auto-compacting the context window"?
- How much token are you paying?

What happens when you type a URL? (1/2)

What Happens When You Type Google.com in Your Browser?

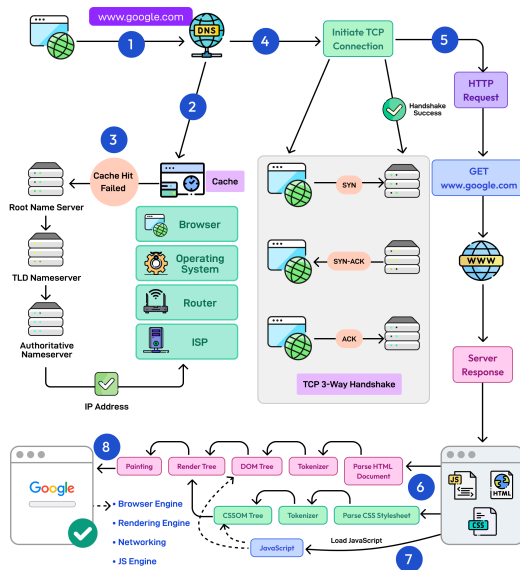


- URL Entry** – You type `www.google.com` in the address bar. The browser parses the URL into protocol (HTTPS), domain, and path.
- Cache Check** – Browser checks its local cache for a recent DNS record. Hit? Skip to step 4. Miss? Proceed to DNS lookup.
- DNS Lookup** – Browser asks the OS, which queries the resolver chain: Root nameserver → TLD nameserver (.com) → Authoritative nameserver → returns the IP address (e.g., 142.250.80.46).
- TCP Connection** – Browser initiates a 3-way handshake with the server: SYN → SYN-ACK → ACK. For HTTPS, a TLS handshake follows to establish encryption.

What happens when you type a URL? (2/2)

What Happens When You Type
Google.com in Your Browser?

ByteByteGo



- HTTP Request & Response** – Browser sends `GET www.google.com`. The server processes the request and returns HTML, CSS, and JS files.
- HTML Parsing** – Browser tokenizes the HTML and builds the DOM tree. It also parses CSS stylesheets into a CSSOM tree.
- JavaScript Execution** – The JS engine (e.g., V8 in Chrome) loads and executes scripts, which may modify the DOM dynamically.
- Rendering & Display** – Browser combines DOM + CSSOM into a render tree, calculates layout (position, size), then paints pixels to the screen. The page appears.

Why does this matter for data systems?

- **Every step involves data representation:** URLs are strings, IP addresses are 32-bit integers (IPv4), HTTP messages are serialized text
- **Every step involves storage:** DNS caches, browser caches, server databases
- **Every step involves networking:** TCP packets, routing, bandwidth
- This course covers the foundations behind all of these

Q: What is a computer?

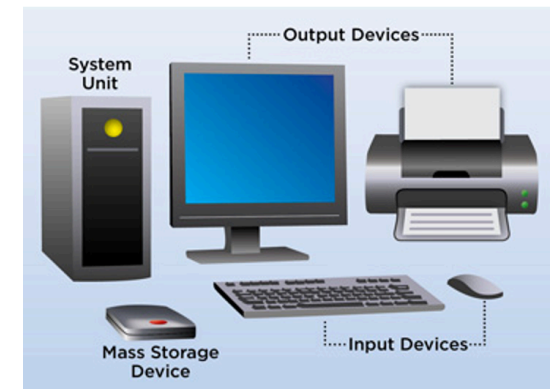
What is a computer?

A programmable electronic device that can **store**, **retrieve**, and **process** digital data.



Basics of Computer Organization

- **Hardware:** The electronic machinery (wires, circuits, transistors, capacitors, devices, etc.)
- **Software:** Programs (instructions) and data



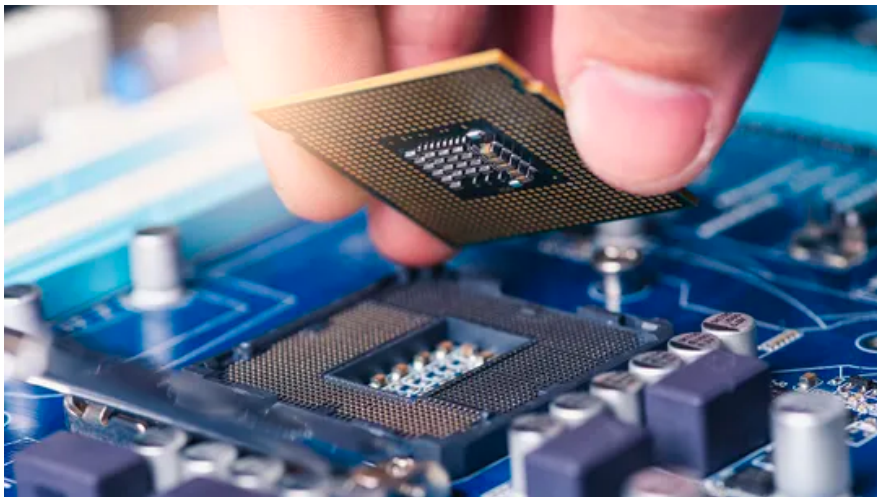
To build a data system, we need:

- **Store and retrieve data:** Disks, Memory
 - Why we need both? (we'll come back to this)
- **Process data:** Processors (CPU and GPU)
- **Retrieve data from remote:** Networks

Key Parts of Computer Hardware

Processor (CPU, GPU, etc.)

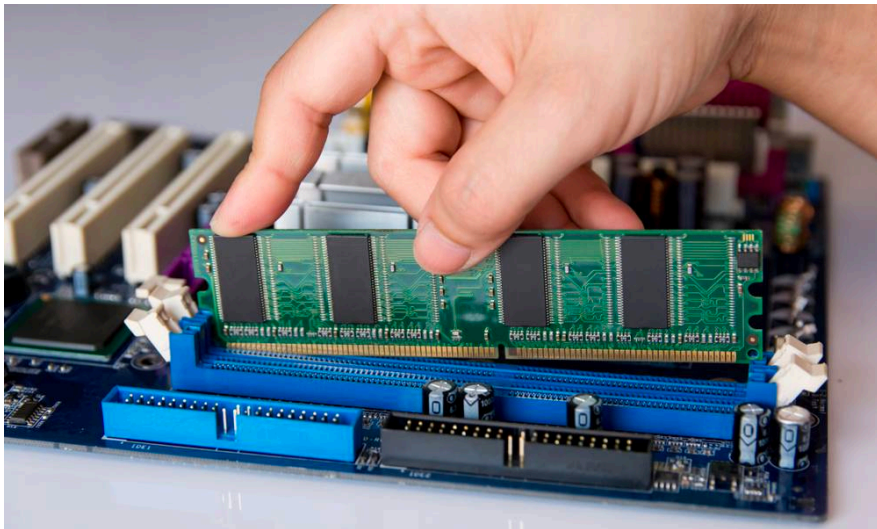
Hardware to orchestrate and execute instructions to manipulate data as specified by a program



Key Parts of Computer Hardware

Main Memory (aka Dynamic Random Access Memory)

Hardware to store data and programs that allows very fast location/retrieval; byte-level addressing scheme



Key Parts of Computer Hardware

Disk (aka secondary/persistent storage)

Similar to memory but persistent, slower, and higher capacity / cost ratio; various addressing schemes



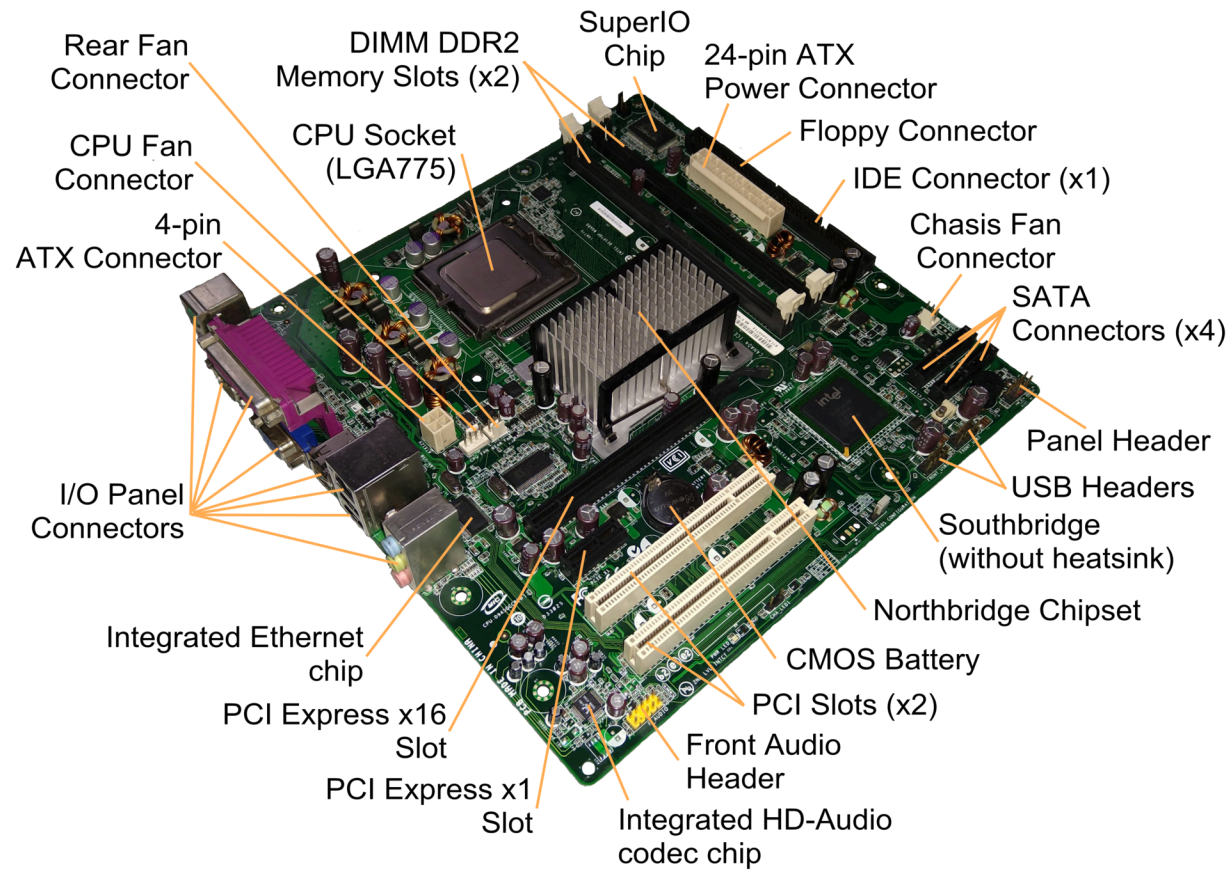
Key Parts of Computer Hardware

Network Interface Controller (NIC)

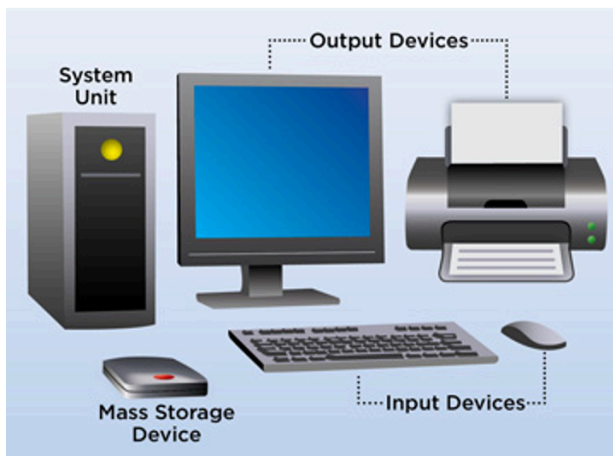
Hardware to send data to / retrieve data over network of interconnected computers/devices



In Reality



Parts of a Computer



- **Hardware:** The electronic machinery (wires, circuits, transistors, capacitors, devices, etc.)
- **Software:** Programs (instructions) and data

Key Aspects of Software

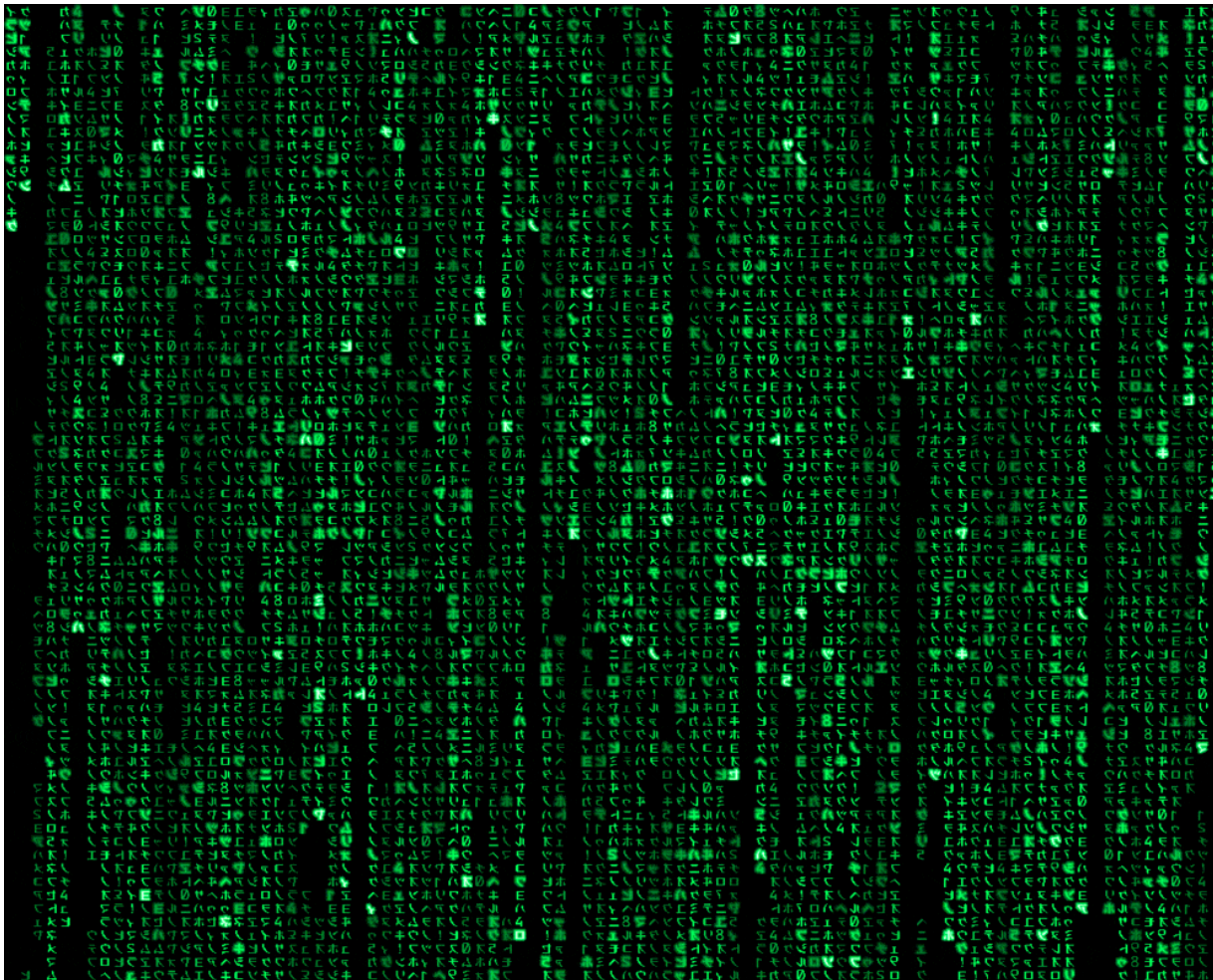
- **Instruction:** A command understood by hardware; finite vocabulary for a processor: Instruction Set Architecture (ISA); bridge between hardware and software
- **Program (aka code):** A collection of instructions for hardware to execute
- **Programming Language (PL):** A human-readable formal language to write programs; at a much higher level of abstraction than ISA
- **API:** A set of functions ("interface") exposed by a program for use by humans/other programs
- **Data:** Digital representation of information that is stored, processed, displayed, retrieved, or sent by a program

Main Kinds of Software

- **Firmware:** Read-only programs "baked into" a device for basic hardware control
- **Operating System (OS):** Collection of interrelated programs that work as an intermediary platform/service to enable application software to use hardware more effectively/easily
 - Examples: Linux, Windows, MacOS, etc.
- **Application Software:** A program or a collection of interrelated programs to manipulate data, typically designed for human use
 - Examples: Excel, Chrome, PostgreSQL, etc.

Data Representation

Q: How is data represented in computers?



Why bits?

- **Physics!** Easy to store with bistable elements
 - e.g., high-low/off-on electromagnetism on disk
- **Reliably transmitted** on noisy and inaccurate wires
- **Very expressive and efficient**
- Quantum computers?

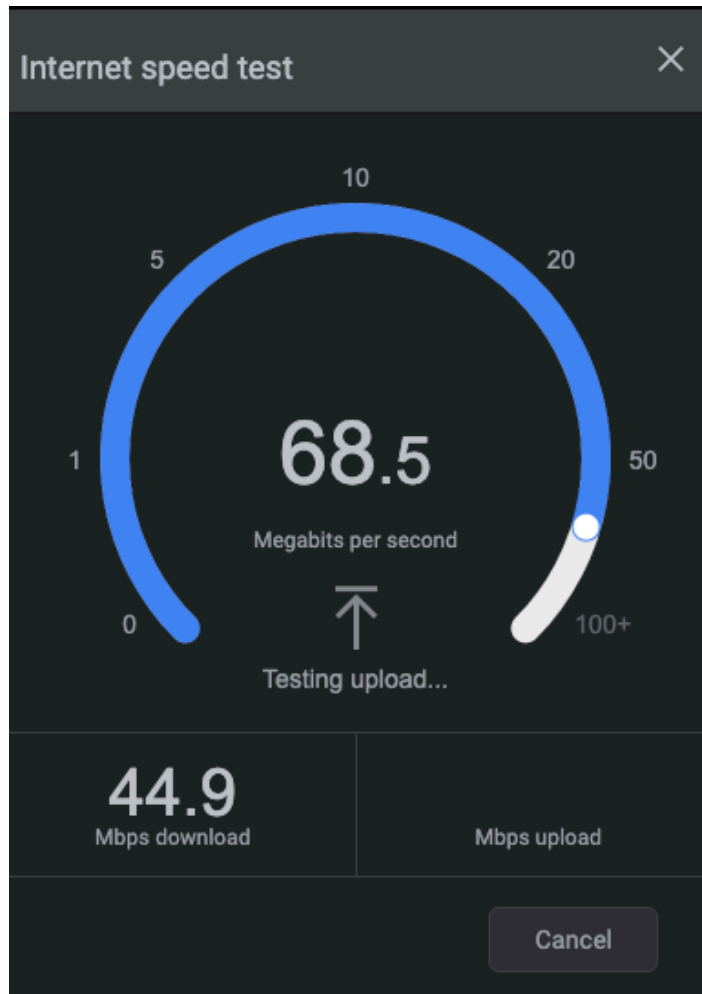
Digital Representation of Data

- **Bits:** All digital data are sequences of 0 & 1 (binary digits)
 - high-low/off-on electromagnetism on disk
- **Data type:** First layer of abstraction to interpret a bit sequence with a human-understandable category of information; interpretation fixed by the PL
 - Example common datatypes: Boolean, Byte, Integer, "floating point" number (Float), Character, and String
- **Data structure:** A second layer of abstraction to organize multiple instances of same or varied data types as a more complex object with specified properties
 - Examples: Array, Linked list, Tuple, Graph, etc.

Count everything in binary

- Use Base 2 to represent Number
 - 0, 1, 10, 11, 100, 101, ...
 - Represent 15213_{10} as $0011\ 1011\ 0110\ 1101_2$
 - Represent 1.20_{10} as $1.0011\ 0011\ 0011\ 0011\ [0011]..._2$
- Represent negative numbers as ...?
 - (we'll come back to this)

(Capital) B (bytes) vs. (lower case) b (bits)



Name	Size ▾	Kind
HB50 cupcakes.JPG	2 MB	JPEG image
Roller Skating.JPG	1.3 MB	JPEG image
50HBJukebox2.jpg	720 KB	JPEG image
Facebook.tiff	399 KB	TIFF image
7_days_to_enrol.png	173 KB	PNG image
JoggingShoes.jpg	71 KB	JPEG image

Encoding Byte Values

- **Byte = 8 bits.** Why?
 - Historical Development
 - Practicality and Standardization
- A Byte (B; 8 bits) is typically the basic unit of data types
 - CPU can't address anything smaller than a byte

Hexadecimal representation

- A common stand-in for binary: more succinct and readable
- Base 16 instead of base 2 – cuts display length by $\sim 4x$
- Digits are 0, 1, ... 9, A (10), B, ... F (15)
- From binary: combine 4 bits at a time from lowest
- Alternative notation: 0xA3 or A3H

Bytes -> Data types: bool, int, float, string, ...

The size and interpretation of a data type depends on PL

- **Boolean:**

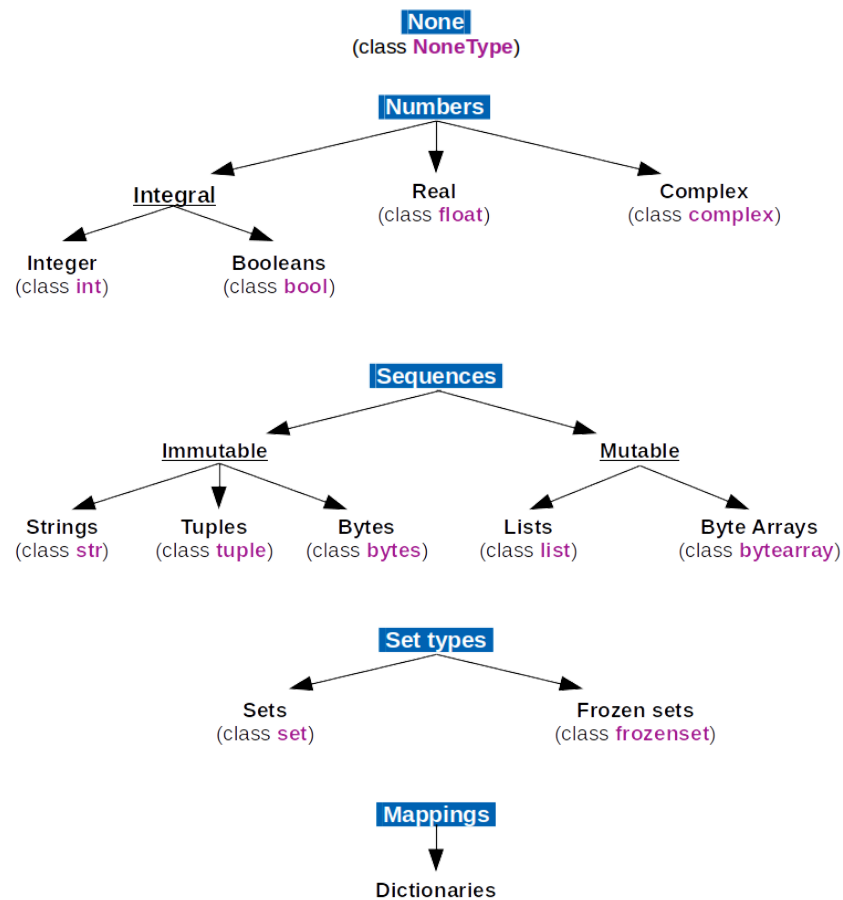
- Examples in data sci.: Y/N or T/F responses
- Just 1 bit needed but actual size is almost always 1B, i.e., 7 bits are wasted!

- **Integer:**

- Examples in data science: #friends, age, #likes
- Typically 4 bytes; many variants (short, unsigned, etc.)
- Java int can represent -2^{31} to $(2^{31} - 1)$; C unsigned int can represent 0 to $(2^{32} - 1)$
- Python3 int is effectively unlimited length (PL magic!)

Data Types in Python 3

Python 3
The standard type hierarchy



How many unique items can k bits represent?

Q: How many unique data items can be represented by 3 bytes?

- Given k bits, we can represent 2^k unique data items
- 3 bytes = 24 bits $\Rightarrow 2^{24}$ items, i.e., **16,777,216** items
- Common approximation: 2^{10} (i.e., 1024) $\sim 10^3$ (i.e., 1000); recall kibibyte (KiB = 1024 B) vs kilobyte (KB = 1000 B)

Q: How many bits are needed to distinguish 97 data items?

- For k unique items, invert the exponent to get $\log_2(k)$
- But #bits is an integer! So, we only need $\lceil \log_2(k) \rceil$
- 97 $\rightarrow 128 = 2^7$; so, **7 bits**

Q: How to convert from decimal to binary?

- Given decimal n , if power of 2 (say, 2^k), put 1 at bit position k ; if $k=0$, stop; else pad with trailing 0s till position 0
- If n is not power of 2, identify the power of 2 just below n (say, 2^k); put 1 at position k
- Reset n as $n - 2^k$; return to Steps 1-2
- Fill remaining positions in between with 0s

Position	7	6	5	4	3	2	1	0
Power of 2	128	64	32	16	8	4	2	1

Can I Run AI locally?

Find out which AI models your machine can actually run.

 Apple M4 Pro *  24 GB *  ~273 GB/s *  24 GB  20

Estimates based on browser APIs. Actual specs may vary. [WebGPU](#)

28 RUNS GREAT | 11 RUNS WELL | 1 DECENT | 0 TIGHT FIT | 18 BARELY RUNS | 19 TOO HEAVY

/ All grades ▾ All tasks ▾ All providers ▾ All licenses ▾ Sort: Score ▾ [List](#)

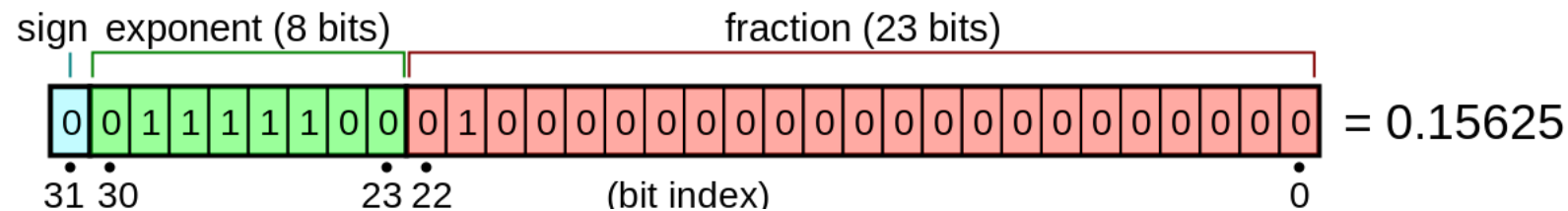
Llama 3.1 8B	  Llama 3.1 Community	1 year ago	4.6 GB	19%	128K ctx	~39 tok/s	RUNS GREAT	85/100
Qwen 3.5 9B	   Apache 2.0	2mo ago	5.1 GB	21%	32K ctx	~35 tok/s	RUNS WELL	82/100
Phi-4 14B	  MIT	1 year ago	7.7 GB	32%	16K ctx	~23 tok/s	RUNS WELL	70/100
GPT-OSS 20B	   Apache 2.0	8mo ago	11.3 GB	47%	128K ctx	~16 tok/s	DECENT	57/100
Mistral Small 3.1 24B	   Apache 2.0	1 year ago	12.8 GB	53%	128K ctx	~14 tok/s	BARELY RUNS	39/100

Float

- Examples in data science: salary, scores, **model weights**
- IEEE-754 single-precision format is **4B** long; double-precision is **8B**
- Java and C float is single; **Python float is double!**

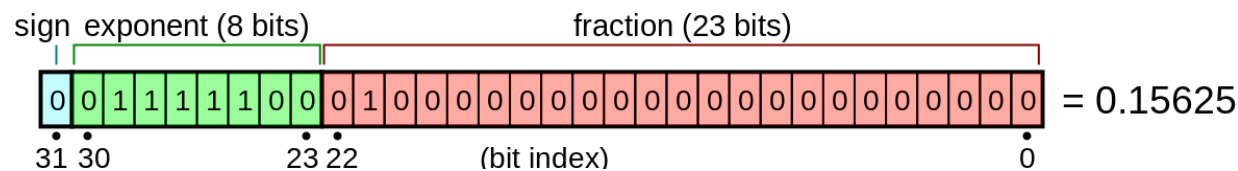
Float: IEEE-754 format

Standard IEEE format for single (aka binary32):



$$(-1)^{sign} \times 2^{exponent-127} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right)$$

Float: IEEE-754 Worked Example

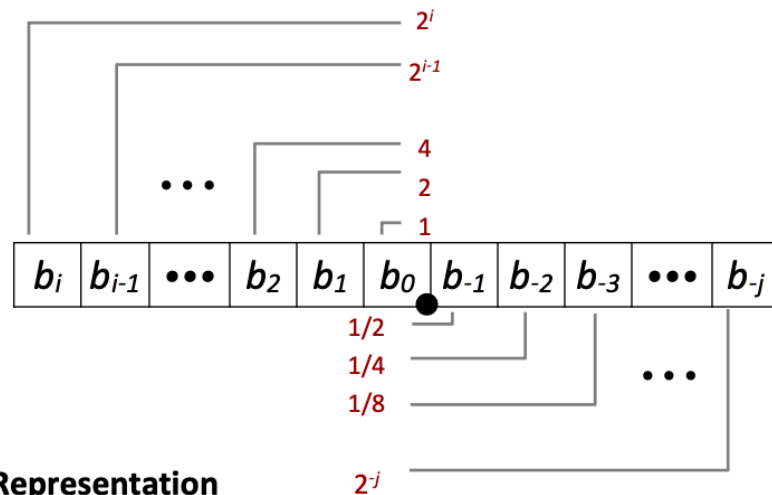


$$(-1)^0 \times 2^{124-127} \times (1 + 1 \cdot 2^{-2})$$

$$= (1/8) \times (1 + (1/4)) = 0.15625$$

Fractional Binary Numbers

Fractional Binary Numbers



■ Representation

- Bits to right of “binary point” represent fractional powers of 2
- Represents rational number:

$$\sum_{k=-j}^i b_k \times 2^k$$

Let's design a fixed-point FP6



$$(-1)^{sign} \times 2^{exponent-2} \times (1 + \sum_{i=1}^3 b_{3-i} 2^{-i})$$

An Example

$$0.625_{10} = 0.625_{10} = 0.101_2$$

An Example (Cont.)

$$0.625_{10} = 0.101_2 = 1.01 \cdot 2^{-1}$$

	sign (1 bit)	exponent (2 bits)		fraction (3 bits)		
	0	0	1	0	1	0
Bit index:	5	4	3	2	1	0

$$(-1)^{sign} \times 2^{exponent-2} \times \left(1 + \sum_{i=1}^3 b_{3-i} 2^{-i}\right)$$

$$(-1)^0 \cdot 2^{(1-2)} \cdot \left(1 + 0 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} + 0 \cdot \frac{1}{8}\right)$$

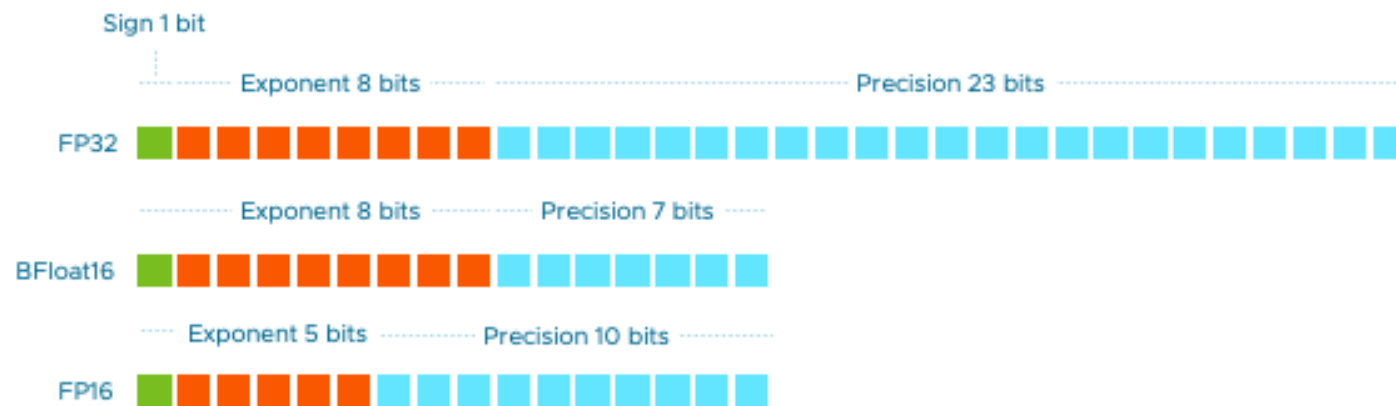
Q: What does exponent and fraction control?

- **Exponent** controls: range, offset
- **Fraction** controls: actual value, precision
- Any problem about floating point (compared to fixed point)?
 - More complex (to both human and computers)
 - **Inconsistent precision**

More float standards

- **double-precision** (float64; 8B) and **half-precision** (float16; 2B); different #bits for exponent, fraction
- **Float16 is now common for deep learning parameters:**
 - Native support in PyTorch, TensorFlow, etc.; APIs also exist for weight quantization/rounding post training

What's the difference between bf16 and fp16?



Fp16 vs. Fp32

- NVIDIA Deep Learning SDK supports **mixed-precision training; 2-3x speedup** with similar accuracy!

Form Factor	H100 SXM
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core	989 teraFLOPS ²
BFLOAT16 Tensor Core	1,979 teraFLOPS ²
FP16 Tensor Core	1,979 teraFLOPS ²
FP8 Tensor Core	3,958 teraFLOPS ²



NVIDIA. DEVELOPER

HOME

BLOG

NEWS

FORUMS

DOCS

DOWNLOADS

TRAINING



ACCOUNT

available on the [NVIDIA deep learning performance page](#).

TRAINING LAYER

AUTOMATIC MIXED PRECISION

ACCELERATED BY GPU

Float: special values and non-associativity

- In binary32, special encodings recognized:
 - Exponent 0xFF and fraction 0 is +/- **"Infinity"**
 - Exponent 0xFF and fraction <> 0 is **"NaN"**
 - Max is $\sim 3.4 \times 10^{38}$; min +ve is $\sim 1.4 \times 10^{-45}$
- **Floating point arithmetic is not associative!**

```
(base) arun@Aruns-MacBook-Pro:~$ python questions/103/103/is-floating-point
Python 3.7.1 (default, Dec 14 2018, 13:28:58)
[Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 0.1 + 0.2
0.30000000000000004
>>> (0.1 + 0.2) + 0.7
1.0
>>> 0.1 + (0.2 + 0.7)
0.9999999999999999
>>>
```

Asked 7 years, 8 months ago Active 1 year, 3 months

I had a problem when I was adding three

Representing Character (char) and String

- Letters, numerals, punctuations, etc.
- A **string** is typically just a variable-sized array of char
 - C char is 1B; Java char is 2B; Python does not have a char type (use str or bytes)
- **American Standard Code for Information Interchange (ASCII)** for encoding characters; initially 7-bit; later extended to 8-bit
 - Examples: 'A' is 65, 'a' is 97, '@' is 64, '!' is 33, etc.
- **Unicode UTF-8** is now common, subsumes ASCII; 4B for ~1.1 million "code points" incl. many other language scripts, math symbols, emojis, etc.

Serialization

Serialization and Deserialization

- A data structure often needs to be **persisted** (stored in a file) or **transmitted** over a network
- **Serialization** is the process of converting program objects into a neat sequence of bytes that can be exactly recovered
- **Deserialization** is the reverse, i.e., bytes back to objects
- For characters/strings: trivial
- Two alternatives for integers/floats:
 - **Byte stream** (aka binary type in SQL): e.g., 4B int => 2B short. Common.
 - **Text format**: CSV, TSV, etc.

Practice Questions

Q1: How much space do I need to store GPT-3?

- What is GPT-3?
 - An ML model with trained weights
 - = a software with some built-in data



Q1: How much space do I need to store GPT-3?

- **Data type?** BF16: 16-bit = **2 bytes**
- **# parameters: 175B**
- **Storage:** 2 bytes x 175B = 350B bytes = **350 GB**

More Practice Qs

- Q1: How much space do I need to store GPT-3?
- Q2: What does exponent and fraction control in float point representation?
- Q3: What is the difference between BF16 and FP16?